

Atelier 3 : boucle Python contre calcul vectorisé

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Chapitre** : 3, Calcul numérique avec NumPy · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	Un notebook qui charge les 51 854 prix au kilo dans des tableaux NumPy, compare la vitesse d'une boucle et du calcul vectorisé, et mesure la hausse du prix du mil dans les 14 régions du Sénégal entre 2015 et 2025
Durée	🟢 Débutant : 3 h 30 · 🔵 Standard : 2 h 30 · 🟣 Expert : 1 h 45 (sans les défis)
Prérequis	Avoir lu le chapitre 3 ; votre dépôt `prix-marches`
Fichiers fournis	`wfp_food_prices_sen.csv` et le notebook de départ
Rendu	Le notebook `atelier3_numpy.ipynb` dans votre dépôt `prix-marches`, avec l'étiquette `atelier-3`

> 📍 **Point de départ** : le notebook de départ fourni. Envoyez le fichier du PAM dans Colab.

Comment travailler

Mêmes règles que les ateliers précédents. **Règle de cet atelier** : à part le chargement (étape 1) et la construction du tableau du mil (étape 6), **aucune boucle** `for` : tout se calcule avec des opérations NumPy.

Étape 1 : les relevés en tableaux NumPy (20 min)

🎯 **Objectif** : quatre tableaux de même longueur, une case par relevé au kilo.

🛠️ **A faire** : en lisant le fichier avec `csv.DictReader`, remplissez quatre listes pour les relevés dont l'unité est `KG` : le prix (`float`), l'année (`int(r["date"][:4])`), la région (`admin1`) et la denrée. Transformez-les en tableaux `prix` , `annees` , `regions` , `denrees` .

✅ **Vérification** : `prix.shape` vaut `(51854,)` ; `annees.dtype` est `int64` ; `regions.dtype` est `<U11` (des textes d'au plus 11 caractères).

💡 **A comprendre** : pourquoi faut-il que les quatre tableaux aient exactement la même longueur, et soient remplis dans le même ordre ?

Étape 2 : les prix en euros (10 min)

🛠️ **A faire** : une constante `PARITE = 655.957` ; le tableau `prix_euros` ; le prix moyen en F CFA et en euros.

✅ **Vérification** : les trois premiers prix `[149. 124.75 227.5]` deviennent `[0.227 0.19 0.347]` euro ; prix moyen 339,6 F CFA, soit 0,518 euro le kilo.

💡 **A comprendre** : ce prix moyen mélange toutes les denrées (du mil aux tomates). A-t-il un sens ?

Étape 3 : sélectionner avec des masques (20 min)

🛠️ **A faire**

1. Le nombre de relevés et le prix moyen du riz importé à Dakar en 2025, avec **un seul** masque combinant trois conditions.
2. Pour le mil (`Millet`) : le nombre de relevés, le prix le plus bas et le plus haut.

✅ **Vérification** : 22 relevés, 458,5 F CFA (le même résultat qu'à l'atelier 2, sans aucune boucle) ; mil : 9 484 relevés, de 100 à 750 F CFA.

Étape 4 : boucle contre calcul vectorisé (30 min)

🎯 **Objectif** : mesurer le gain de NumPy sur un vrai calcul statistique, la **standardisation** (le z-score).

Le **z-score** d'un prix dit de combien d'écarts-types il s'éloigne de la moyenne : z = (prix - moyenne) / écart-type. Un z de 2 veut dire « deux écarts-types au-dessus de la moyenne ».

🛠️ **A faire**

1. **En Python pur**, sur `liste = prix.tolist()` : la moyenne, l'écart-type (racine carrée de la moyenne des carrés des écarts), puis la liste des z-scores. Chronométrez avec `time.perf_counter()` .
2. **Avec NumPy**, en une ligne : `(prix - prix.mean()) / prix.std()` . Chronométrez aussi.
3. Vérifiez que les deux donnent les mêmes résultats avec `np.allclose(z_boucle, z_numpy)` .

✅ **Vérification** : `Mêmes résultats : True` ; NumPy est environ 10 à 30 fois plus rapide (le rapport dépend de l'ordinateur et change à chaque exécution).

💡 **A comprendre** : pourquoi `np.allclose` et pas `==` pour comparer deux résultats de calcul à virgule ?

Étape 5 : les valeurs extrêmes (15 min)

🛠️ **A faire** : comptez les relevés dont le z-score dépasse 3 en valeur absolue (`np.abs`), et affichez la liste triée des denrées concernées (`sorted({str(d) for d in denrees[extremes]})`).

✅ **Vérification** : 1 525 relevés ; 14 denrées, des bananes aux pastèques (`Watermelons`), en passant par les tomates, les arachides et le riz local.

💡 **A comprendre** : ces 1 525 relevés sont-ils des **erreurs** ? Pourquoi calculer un z-score sur toutes les denrées à la fois n'a pas beaucoup de sens ? Comment faudrait-il faire ?

Étape 6 : le mil, région par région (35 min)

🎯 **Objectif** : un tableau à deux dimensions, 14 régions x 11 années.

🛠️ **A faire**

1. `liste_regions` (les régions triées) et les années de 2015 à 2025.
2. Deux tableaux `sommes` et `comptes` de forme `(14, 11)` remplis de zéros (`np.zeros` et `np.zeros_like`), puis **une** boucle sur les relevés de mil de ces années, comme dans la section 7 du chapitre.
3. La division, **sans erreur si une case est vide** : `np.divide(sommes, comptes, out=np.full_like(sommes, np.nan), where=comptes > 0)` .
4. Pour chaque région, le prix moyen du mil en 2015 et en 2025.

✅ **Vérification** : forme `(14, 11)` , 0 case vide ; Dakar 242,3 F en 2015 et 292,0 F en 2025 ; Kedougou 292,2 et 414,6 ; Kaffrine 171,3 et 201,1.

💡 **A comprendre** : à quoi servent `out=` et `where=` ? Que donnerait une simple division `sommes / comptes` s'il y avait une case vide ?

Étape 7 : l'indice base 100 (25 min)

🛠️ **A faire**

1. L'indice base 100 en 2015 pour chaque région, **par broadcasting** : `mil_moyen / mil_moyen[:, [0]] * 100` (remarquez les doubles crochets).
2. Avec `np.argsort` sur la dernière colonne, la région où le mil a le plus augmenté entre 2015 et 2025, et celle où il a le moins augmenté ; l'indice moyen national.

✅ **Vérification** : plus forte hausse : **Tambacounda** (indice 167) ; plus faible : **Kaffrine** (117) ; indice moyen : 139.

💡 **A comprendre**

- `mil_moyen[:, [0]]` a la forme `(14, 1)`, alors que `mil_moyen[:, 0]` a la forme `(14,)`. Pourquoi la première fonctionne-t-elle pour le broadcasting ?
- Kaffrine est une grande région de production de céréales. Cela peut-il expliquer la faible hausse ? Quelle autre information faudrait-il pour le vérifier ?

Puis : **Exécution > Redémarrer la session et tout exécuter**, déposez le notebook dans votre dépôt, et étiquetez `atelier-3`.

🏆 Défis (facultatifs)

1. **Un z-score juste.** Calculez le z-score de chaque relevé **par rapport à sa propre denrée** (moyenne et écart-type de la denrée). Combien de relevés dépassent maintenant 3 ?
2. **Le riz importé, région par région.** Refaites les étapes 6 et 7 pour le riz importé. Les régions qui ont le plus augmenté sont-elles les mêmes que pour le mil ?
3. **Les mois.** Ajoutez un tableau des **mois** au chargement, et calculez le prix moyen du mil pour chaque mois de l'année (toutes années confondues). Les prix sont-ils plus élevés pendant la période de soudure (de juin à septembre, avant les récoltes) ?
4. 🟣 **Sans boucle du tout.** Construisez le tableau de l'étape 6 **sans** boucle `for`, avec `np.unique(..., return_inverse=True)` pour transformer régions et années en numéros, puis `np.add.at`.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
transformer des données en tableaux NumPy et lire leur forme et leur type	☐	☐
calculer sur tout un tableau sans boucle	☐	☐
sélectionner avec un masque à plusieurs conditions	☐	☐
mesurer et expliquer le gain de vitesse de NumPy	☐	☐
construire un tableau à deux dimensions et calculer par axe	☐	☐
utiliser le broadcasting pour un indice base 100	☐	☐
critiquer un calcul statistique qui mélange des choses différentes	☐	☐